

系统架构设计师重要知识点 100 条

1、使用许可

按照被许可使用权的排他性强弱不同，可以将使用许可分为以下三种：

独占使用许可-仅 1 个授权对象可用，著作权人不可用

排他使用许可-仅 1 个授权对象和著作权人可用

普通使用许可-多个授权对象和著作权人可用

2、企业集成分类：按集成点分

	集成点	效果	解题关键点
界面集成	界面	统一入口，产生“整体”感觉	“整体”感觉 最小代价实现一体化操作
数据集成	数据	不同来源的数据逻辑或物理上“集中”	其他集成方法的基础
控制集成	应用逻辑	调用其他系统已有方法，达到集成效果	
业务流程集成 (过程集成)	应用逻辑	跨企业，或优化流程而非直接调用	企业之间的信息共享能力
门户集成		将内部系统对接到互联网上	发布到互联网上

3、EAI 提供 4 个层次的服务（从高层到低层）：

EAI 提供 4 个层次的服务	功能
流程控制服务	解决人工参与的长期的工作流程控制问题
应用连接服务	应用连接适配器将应用接口连接至 EAI 平台
信息传递与转化服务	负责传递消息和转化消息
通讯服务	通过通讯中间件进行消息的路由

4、电子政务

政府对政府（G2G，Government To Government）：政府之间的互动及政府与公务员之间

互动。基础信息的采集、处理和利用，如人口/地理/资源信息等；各级政府决策支持；政府间通信。

政府对企业（G2B, Government To Business）：政府为企业提供的政策环境。包括产业政策、进出口、注册、纳税、工资、劳保、社保等各种规定；政府向企事业单位颁发的各种营业执照、许可证、合格证、质量认证等。

企业对政府（B2G, Business To Government）：企业纳税及企业为政府提供服务。企业参加政府各项工程的竞/投标，向政府提供各种商品和服务，企业向政府提建议，申诉。

政府对公民（G2C, Government To Citizen）：政府对公民提供的服务。关于社区公安和水、火、天灾等与公共安全有关的信息等，还包括户口、各种证件的管理等政府提供的各种服务。

公民对政府（C2G, Citizen To Government）：个人应向政府缴纳税费和罚款及公民反馈渠道。个人应向政府缴纳的各种税款和费用。了解民意，征求群众意见。报警服务（盗贼、医疗、急救、火警等）。

政府对公务员（G2E, Government To Employee）：政府与政府公务员即政府雇员。政府机构通过网络技术实现内部电子化管理（例如，OA 系统等）的重要形式。

5、逆向工程及其相关的概念

与逆向工程相关的概念有重构、设计恢复、再工程和正向工程。

（1）重构（restructuring）。重构是指在同一抽象级别上转换系统描述形式。

（2）设计恢复（design recovery）。设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

（3）逆向工程（reverse engineering）：逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是设计的恢复过程。

（4）正向工程（forward engineering）。正向工程是指不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

（5）再工程（re-engineering）。再工程是对现有系统的重新开发过程，包括逆向工程、新需求的考虑过程和正向工程三个步骤。

6、结构化开发方法

自顶向下，逐步分解（求解），严格区分工作阶段，每阶段有任务与成果，强调系统开发过程的整体性和全局性，开发目标清晰化，工作阶段程式化，开发文档规范化，设计方法结构化。

优点：理论基础严密，它的指导思想是用户需求在系统建立之前就能被充分了解和理解。由此可见，结构化方法注重开发过程的整体性和全局性。

缺点：开发周期长；文档、设计说明繁琐，工作效率低。

7、原型法开发方法

原型法是一种根据用户需求，利用系统开发工具，快速地建立一个系统模型展示给用户，在此基础上与用户交流，最终实现用户需求的信息系统快速开发的方法。适用于需求不明确的开发。

从原型是否实现功能来分，可以分为水平原型和垂直原型：

水平原型也称为行为原型，用来探索预期系统的一些特定行为，并达到细化需求的目的。水平原型通常只是功能的导航，但未真实实现功能。水平原型主要用在界面上。

垂直原型也称为结构化原型，实现了一部分功能。垂直原型主要用在复杂的算法实现上。

从原型的最终结果来分，可分为抛弃式原型和演化式原型：

抛弃式原型也称为探索式原型，是指达到预期目的后，原型本身被抛弃。抛弃式原型主要用在解决需求不确定性、二义性、不完整性、含糊性等。

演化式原型为开发增量式产品提供基础，逐步将原型演化成最终系统，主要用在必须易于升级和优化的场合，适合于 Web 项目。

8、面向对象方法

最早来源于仿真领域，其特点是系统的描述及信息模型的表示与客观实体相对应，符合人们的思维习惯，有利于系统开发过程中用户与开发人员的交流和沟通，缩短开发周期，提供系统开发的准确性和效率。具有更好的复用性，关键在于建立一个全面、合理、统一的模型，分析、设计、实现三个阶段界限不明确。

OMT 是面向对象建模技术（UML 前身）。用 OMT 方法开发软件，通常需要建立三种形式的模型：对象模型（描述系统数据结构）、动态模型（描述系统控制结构）、功能模型（描述系统功能）。

9、面向服务的方法

以粗粒度、松散耦合的系统功能为核心，强调系统功能的标准化和构件化，加强了系统的灵活性、可复用性和可演化性。

从概念上讲，SOA 中有三个主要的抽象级别：

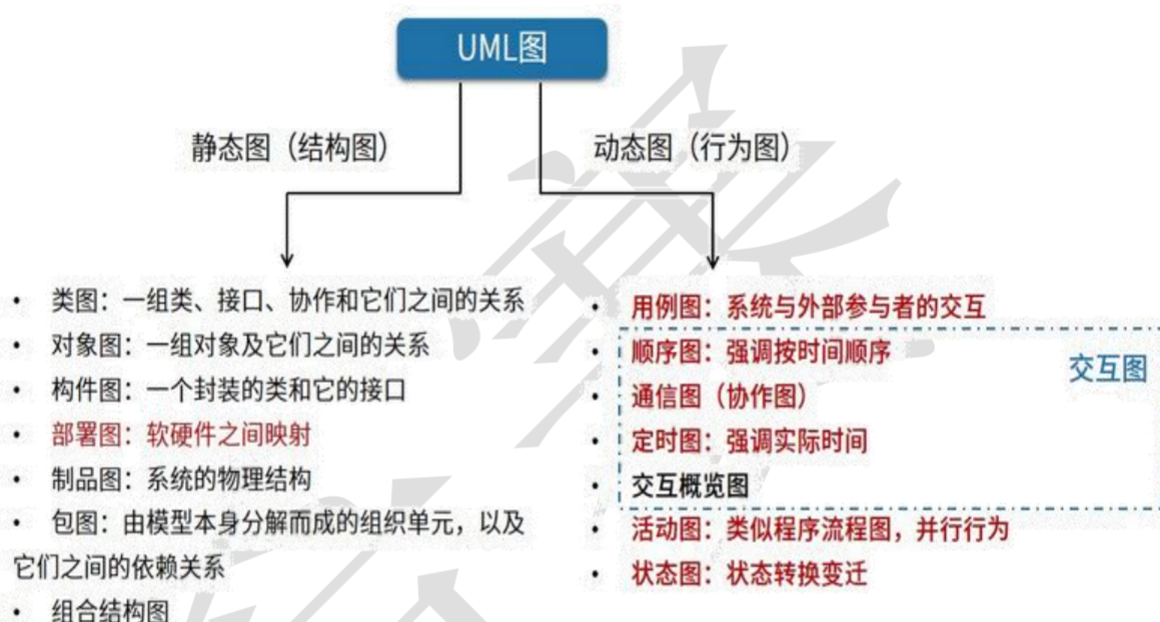
操作：代表单个逻辑工作单元（LUW）的事务。执行操作通常会导致读、写或修改一个或

多个持久性数据。SOA 操作可以直接与面向对象 (OO) 的方法相比。它们都有特定的结构化接口，并且返回结构化的响应。操作位于最底层。

服务：代表操作的逻辑分组。例如，如果我们将 CustomerProfiling 视为服务，则按照电话号码查找客户、按照名称和邮政编码列出顾客和保存新客户的数据就代表相关的操作。

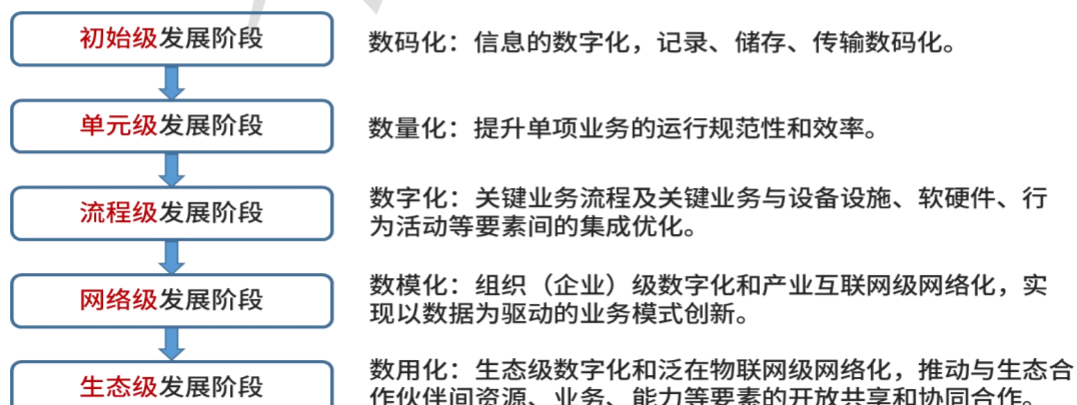
业务流程：为实现特定业务目标而执行的一组长期运行的动作或活动。业务流程通常包括多个业务调用。业务流程的例子有：接纳新员工、出售产品或服务 and 完成订单。

10、UML 图分类



11、数字化转型的 5 个发展阶段

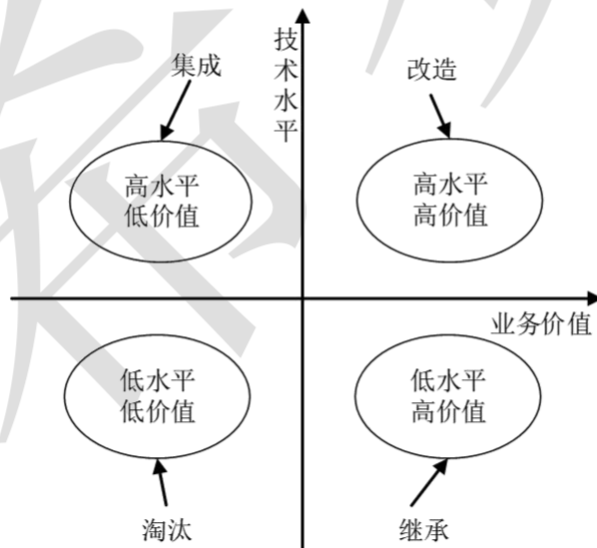
企业数字化转型的五个发展阶段分别为：初始级发展阶段、单元级发展阶段、流程级发展阶段、网络级发展阶段、生态级发展阶段。



12、信息系统的分类

信息系统的分类	关键点
业务处理系统 【TPS】	早期最初级的信息系统【20 世纪 50-60 年代】。 功能：数据输入、数据处理【批处理、OLTP】、数据库维护、文件报表产生。
管理信息系统 【MIS】	高度集成化的人机信息系统。 金字塔结构：分多个层级。
决策支持系统 【DSS】	由语言系统、知识系统和问题处理系统组成。 用于辅助决策、支持决策。
专家系统【ES】	知识 + 推理 = 专家系统。人工智能的一个重要分支。
办公自动化系统 【OAS】	由计算机设备、办公设备、数据通信及网络设备、软件系统组成。
企业资源计划 【ERP】	打通供应链，集成，整合。

13、遗留系统演化策略



淘汰策略：遗留系统的技术含量较低，且具有较低的业务价值。对遗留系统的完全淘汰是企业资源的根本浪费，系统分析师应该善于“变废为宝”，通过对遗留系统功能的理解和借鉴，可以帮助新系统的设计，降低新系统开发的风险。

继承策略：遗留系统的技术含量较低，已经满足企业运作的功能或性能要求，但具有较高的商业价值，目前企业的业务尚紧密依赖该系统。对这种遗留系统的演化策略为继承。在开发新系统时，需要完全兼容遗留系统的功能模型和数据模型。为了保证业务的连续性，新老系

统必须并行运行一段时间，再逐渐切换到新系统上运行。

改造策略：遗留系统具有较高的业务价值，基本上能够满足企业业务运作和决策支持的需要。这种系统可能建成的时间还很短，对这种遗留系统的演化策略为改造。改造包括系统功能的增强和数据模型的改造两个方面。系统功能的增强是指在原有系统的基础上增加新的应用要求，对遗留系统本身不做改变；数据模型的改造是指将遗留系统的旧的数据模型向新的数据模型的转化。

集成策略：遗留系统的技术含量较高，但其业务价值较低，可能只完成某个部门（或子公司）的业务管理。这种系统在各自的局部领域里工作良好，但对于整个企业来说，存在多个这样的系统，不同的系统基于不同的平台、不同的数据模型，形成了一个信息孤岛，对这种遗留系统的演化策略为集成。

14、新旧系统转换策略

直接转换：直接转换是在原有系统停止运行的某一时刻，新系统立即投入运行，中间没有过渡阶段。采用这种方式时，人力和费用最省，适用于系统不太复杂或现有系统完全不能使用的场合。但是这种方式风险高。

并行转换：并行转换就是新系统和旧系统并行工作一段时间，经过这段时间的试运行后，再用新系统正式替换下现有系统。那么这种方式，它的好处就是风险很小。在转换期间还可以同时比较新旧两套系统的性能，而且能够让操作人员得到全面的培训，所以对于一些比较大的信息系统，或者处理过程比较复杂，数据比较重要的系统。并行转换是一种最常用的转换方式。那么这种转换方式也有缺点，缺点就在于两套系统并行期间，要有两套班子或者两套处理方式同时并存，在人力和费用消耗比较大，转换的周期比较长，而且难以控制新旧系统当中数据的变化。所以这就要求要做好转换计划，并且要加强管理。

分段转换：这是直接转换和并行转换的结合，也就是分期分批、逐步转换。一般比较大的系统采用这种方式比较合适，它能够保证软件平稳运行，费用也不太高，就是将大的系统分成多个子系统，每成熟一个子系统就切换一个子系统，主要是分期分批。这种分段转换的策略，它的优点就是成熟一个子系统就转换一个子系统。这种新旧转换，震动比较小，用户比较容易接受。但是由于采取的是渐进的方式，会导致新旧系统的转换周期比较长。

15、系统维护分类

正确性维护：指改正在系统开发阶段已发生而系统测试阶段尚未发现的错误。

适应性维护：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。

完善性维护：为扩充功能和改善性能而进行的修改。

预防性维护：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使用系统适应各类变化而不被淘汰。如将专用报表功能改成通用报表生成功能，以适应将来报表格式的变化。

16、智能制造

智能制造体系架构中，系统层级是指与企业生产活动相关的组织结构的层级划分，包括设备层、单元层、车间层、企业层和协同层。

- (1) 设备层是指企业利用传感器、仪器仪表、机器、装置等，实现实际物理流程并感知和操控物理流程的层级；
- (2) 单元层是指用于企业内处理信息、实现监测和控制物理流程的层级；
- (3) 车间层是实现面向工厂或车间的生产管理的层级；
- (4) 企业层是实现面向企业经营管理的层级；
- (5) 协同层是企业实现其内部和外部信息互联和共享，实现跨企业间业务协同的层级。

17、基于构件的软件工程（CBSE）

CBSE 体现了“购买而不是重新构造”的哲学。

CBSE 的构件应该具备的特征：

可组装性：所有外部交互必须通过公开定义的接口进行。

可部署性：构件总是二进制形式的，能作为一个独立实体在平台上运行。

文档化：用户根据文档来判断构件是否满足需求。

独立性：可以在无其他特殊构件的情况下进行组装和部署。

标准化：符合某种标准化的构件模型。

【构件的组装】：

顺序组装：按顺序调用已经存在的构件，可以用两个已经存在的构件来创造一个新的构件。

层次组装：被调用构件的“提供”接口必须和调用构件的“请求”接口兼容。

叠加组装：多个构件合并形成新构件，新构件整合原构件的功能，对外提供新的接口。

18、构件模型要素

【接口】构件通过构件接口来定义，构件模型规定应如何定义构件接口以及在接口定义中应该包含的要素，如操作名、参数以及异常等。

【使用信息】为使构件远程分布和访问，必须给构件一个特定的、全局唯一的名字或句柄。构件元数据是构件本身相关的数据，比如构件的接口和属性信息。用户可以通过元数据找到

构件提供的服务。构件模型的实现通常包括访问构件的元数据的特定方法。构件是通用实体，在部署的时候，必须对构件进行配置来适应应用系统。

【部署】构件模型包括一个规格说明，指出应该如何打包构件使其部署成为一个独立的 可执行实体。部署信息中包含有关包中内容的信息和它的二进制构成的信息。

19、净室软件工程

净室即无尘室、洁净室。也就是一个受控污染级别的环境。

使用盒结构规约（或形式化方法）进行分析和设计建模，并且强调将正确性验证，而不是测试，作为发现和消除错误的主要机制。

使用统计的测试来获取认证被交付的软件的可靠性所必需的出错率信息。

【技术手段】：

(1) 统计过程控制下的增量式开发：控制迭代

(2) 基于函数的规范和设计：盒子结构

定义 3 种抽象层次：行为视图（黑盒）→→有限状态机视图（状态盒）→→过程视图（明盒）

(3) 正确性验证：净室工程的核心

(4) 统计测试和软件认证：使用统计学原理，总体太大时必须采用抽样方法。

【缺点】：

太理论化，正确性验证的步骤比较困难且耗时。

开发小组不进行传统的模块测试，这是不现实的。

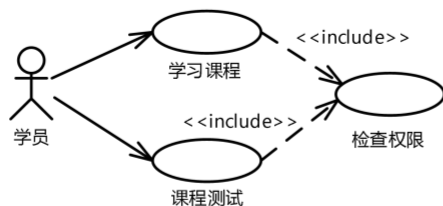
脱胎于传统软件工程，不可避免带有传统软件工程的一些弊端。

20、数据流图 DFD

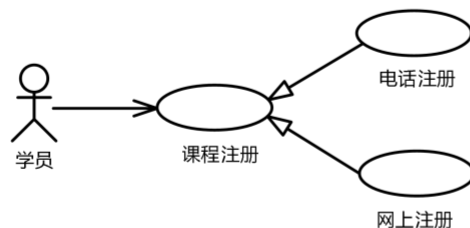
元素	说明	图元
数据流	由一组固定成分的数据组成，表示数据的流向。 每个数据流通常有一个合适的名词，反映数据流的含义	
加工	加工描述了输入数据流到输出数据流之间的变换，也就是输入数据流做了什么处理后变成了输出数据流	
数据存储 (文件)	用来表示暂时存储的数据，每个文件都有名字。 流向文件的数据流表示写文件，流出的表示读文件	
外部实体	指存在于软件系统外的人员或组织	

21、UML 图关系

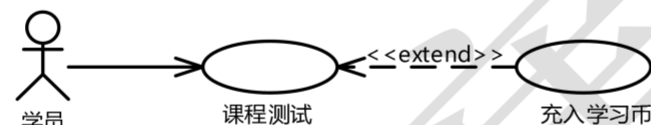
用例关系包括：包含关系、扩展关系、泛化关系。



包含关系



泛化关系



扩展关系

包含关系：其中这个提取出来的公用用例称为抽象用例，而把原始用例称为基本用例或基础用例，当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们。

扩展关系：如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样使描述可能更加清晰。

泛化关系：当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。在用例的泛化关系中，子用例是父用例的一种特殊形式，子用例继承了父用例所有的结构、行为和关系。

22、原型模型

典型的原型开发方法模型。适用于需求不明确的场景，可以帮助用户明确需求。可以分为【抛弃型原型】与【演化型原型】

原型模型两个阶段：

- (1) 原型开发阶段；
- (2) 目标软件开发阶段。

23、瀑布模型

瀑布模型是将软件生存周期中的各个活动规定为线性顺序连接的若干阶段的模型，包括需求分析、软件设计、程序设计、编码实现、单元测试、集成测试、系统测试、运行维护。

瀑布模型的特点是严格区分阶段，每个阶段因果关系紧密相连，只适合需求明确的项目。

24、增量模型

融合了瀑布模型的基本成分和原型实现的迭代特征，可以有多个可用版本的发布，核心功能往往最先完成，在此基础上，每轮迭代会有新的增量发布，核心功能可以得到充分测试。强调每一个增量均发布一个可操作的产品。

25、V 模型和 W 模型

V 模型强调测试贯穿项目始终，而不是集中在测试阶段。是一种测试的开发模型。

W 模型强调测试和开发【并行进行】。

26、敏捷开发

敏捷开发是一种以人为核心、迭代、循序渐进的开发方法，适用于小团队和小项目，具有小步快跑的思想。常见的敏捷开发方法有极限编程法、水晶法、并列争球法和自适应软件开发方法。

27、统一过程（在软考中 UP、RUP 都指统一过程）

典型特点是用例驱动、以架构为中心、迭代和增量。

统一过程把一个项目分为四个不同的阶段：

构思阶段（初始/初启阶段）：定义最终产品视图和业务模型、确定系统范围。

细化阶段（精化阶段）：设计及确定系统架构、制定工作计划及资源要求。

构造阶段：开发剩余构件和应用程序功能，把这些构件集成为产品，并进行详细测试。

移交阶段：确保软件对最终用户是可用的，进行β测试，制作产品发布版本。

9 个核心 workflow：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。

28、系统建模方法

（1）结构化建模方法：结构化建模方法是以过程为中心的技术，可用于分析一个现有的系统以及定义新系统的业务需求。结构化建模方法所绘制的模型称为数据流图（DFD）。对于流程较为稳定的系统可考虑结构化建模方法。

（2）信息工程建模方法（或数据库建模方法）：信息工程建模方法是一种以数据为中心，但过程敏感的技术，它强调在分析和研究过程需求之前，首先研究和分析数据需求。信息工程

建模方法所创建的模型被称为实体-联系图（ERD）。主要用于数据建模。

（3）面向对象建模方法：面向对象建模方法将“数据”和“过程”集成到被称为“对象”的结构中，消除了数据和过程的人为分离现象。面向对象建模方法所创建的模型被称为对象模型。随着面向对象技术的不断发展和应用，形成了面向对象的建模标准，即 UML（统一建模语言）。UML 定义了几种不同类型的模型图，这些模型图以对象的形式共建一个信息系统或应用系统。目前比较常用的建模方法。

29、MBSE（Model Based Systems Engineering，基于模型的系统工程）

MBSE 是建模方法的形式化应用，以使建模方法支持系统需求、分析、设计、验证和确认等活动，这些活动从概念性设计阶段开始，持续贯穿到设计开发以及后来的所有生命周期阶段。

MBSE 的三大支柱：

建模语言：沟通标准化，MBSE 的核心。典型代表:SysML，OPM。

建模思路：类似于路线图，如:INCONSE 面向对象系统工程法。

建模工具：如:Agilian，Modelio。

30、SysML 中的需求关系

SysML 中的需求关系：包含、跟踪、继承需求、改善、满足、验证和复制。

包含（Include）：需求可以且只能包含其他需求。

跟踪（Trace）：对提供方元素（位于箭头端）的修改可能会导致对客户端元素（位于尾端）修改的需要。

继承（deriveReq）：一个需求可以继承另一个需求的属性。

改善（refine）：表示一个需求改进了另一个需求的满足程度。

满足（satisfy）：一般是模块满足某种需求。

验证（verify）：表示一个需求验证了另一个需求的正确性。

复制（Copy）：表示一个需求复制了另一个需求的特性。

31、人机界面设计—“黄金三准则”

- （1）置于用户控制之下
- （2）减少用户的记忆负担
- （3）保持界面的一致性

32、测试阶段划分

测试阶段	内容
单元测试	依据详细设计，模块测试，模块功能、性能、接口等
集成测试	依据概要设计，模块间的接口
系统测试	依据需求文档，在真实环境下，验证完整的软件配置项能否和系统正确连接
确认测试	依据需求文档，验证软件与需求的一致性。内部确认测试、Alpha 测试、Beta 测试、验收测试

33、白盒测试

概念：根据内部结构和逻辑来设计测试用例，对程序路径和过程进行测试。主要用于单元测试阶段。

	定义	特点
语句覆盖	被测试程序中的每条语句至少执行一次。	对执行逻辑覆盖很低，一般认为是很弱的逻辑覆盖。
判定覆盖 (分支覆盖)	被测程序每个判定表达式至少获得一次“真”值和“假”值（或者程序中每一个判定取“真”分支和取“假”分支至少通过一次。）	判定覆盖比语句覆盖更强一些。 判定可以是 1 个条件，也可以是多个条件的组合。
条件覆盖	每一个判定语句中每个逻辑条件的各种可能的值至少满足一次。	条件覆盖和判断覆盖没有包含关系。
判断/条件覆盖	判定中每个条件的所有可能取值（真/假）至少出现一次，并使每个判定本身的判定结果（真/假）也至少出现一次。	同时满足判定覆盖和条件覆盖
条件组合覆盖	每个判定中的各种可能值的组合都至少出现一次。	同时满足判定覆盖、条件覆盖、判定/条件覆盖。
路径覆盖	覆盖被测试程序中所有可能的路径。	
基本路径测试	每一条独立路径都执行过（即程序中可执行语句至少执行一次）。	测试用例个数与环路复杂度一致。判定为关键控制结点，必须出现在基本路径中。
循环覆盖	循环中每个条件都得到验证。	注意数组参数可循环验证

34、黑盒测试（功能测试）

概念：黑盒测试基于产品功能规格说明书，从用户角度针对产品特定的功能和特性来进行验

证活动，确认每个功能是否得到完整实现，用户能否正常使用这些功能。主要用于集成测试、确认测试和系统测试阶段。

黑盒测试在不知道系统或组件内部结构的情况下进行，不考虑内部逻辑结构，着眼于程序外部结构，在软件接口处进行测试。

试图发现的错误：功能不正确或遗漏；界面错误；数据库访问错误；性能错误；初始化和终止错误等

方法：等价类划分法；边界值分析法；因果图法；判定表驱动法；正交试验设计法；错误推测法；功能图法。

等价类划分：不同等价类，揭示不同问题；有效等价类/无效等价类。
边界值分析： $1 \leq x \leq 10$ ，可取 x 的值为 0、1、10 和 11 作为测试数据
错误推测：依靠测试人员的经验和直觉。
判定表：最适合描述在多个逻辑条件取值的组合所构成的复杂情况下，分别要执行哪些不同的动作。
因果图：根据输入条件与输出结果之间的因果关系来设计测试用例。

35、其他软件测试方法

测试方法	描述
AB 测试	多版本同时使用，利于收集各版本的用户反馈，评估出最好版本。故也算是一种【网页优化方法】。
Web 测试	Web 系统测试与其他系统测试测试内容基本相同，只是测试重点不同。 Web 代码测试包括：源代码规则分析、链接测试、框架测试、表格测试、图形测试等方面。
链接测试	链接测试可分为 3 个方面： 1、测试所有链接是否按指示的那样确实链接到了该链接的页面。 2、测试所链接的页面是否存在。 3、保证 Web 应用系统上没有孤立的页面。
表单测试	验证服务器是否能正确保存这些数据，后台运行的程序能否正确解释和使用这些信息。测试提交操作的完整性。
回归测试	测试软件变更之后，变更部分的正确性和对变更需求的符合性。

36、模块独立性的度量——聚合

聚合：衡量模块内部各元素结合的紧密程度。

偶然聚合：模块完成的动作之间没有任何关系，或者仅仅是一种非常松散的关系。

逻辑聚合：模块内部的各个组成在逻辑上具有相似的处理动作，但功能用途上彼此无关。

时间聚合：模块内部的各个组成部分所包含的处理动作必须在同一时间内执行。

过程聚合：模块内部各个组成部分所要完成的动作虽然没有关系，但必须按特定的次序执行。

通信聚合：模块的各个组成部分所完成的动作都使用了同一个数据或产生同一输出数据。

顺序聚合：模块内部的各个部分，前一部分处理动作的最后输出是后一部分处理动作的输入。

功能聚合：模块内部各个部分全部属于一个整体，并执行同一功能，且各部分对实现该功能都必不可少

37、模块独立性的度量——耦合

耦合：度量不同模块间互相依赖的程度。

非直接耦合：两个模块之间没有直接关系，它们的联系完全是通过主模块的控制和调用来实现的。

数据耦合：两个模块彼此间通过数据参数交换信息。

标记耦合：一组模块通过参数表传递记录信息，这个记录是某一个数据结构的子结构，而不是简单变量。

控制耦合：两个模块彼此间传递的信息中有控制信息。

外部耦合：一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息。

公共耦合：两个模块之间通过一个公共的数据区域传递信息。

内容耦合：一个模块需要涉及到另一个模块的内部信息。

38、软件过程改进 CMMI

成熟度等级	过程域	关键特征
优化级【L5】	组织级改革与实施、因果分析和解决方案	持续优化
定量管理级【L4】	组织级过程性能、定量项目管理	量化式管理
已定义级【L3】	需求开发、技术解决方案、产品集成、验证、确认、组织级过程焦点、组织级过程定义、组织级培训、集成项目管理、风险管理、集成化的团队、决策分析和解决方案、组织级集成环境	组织级，文档化标准化
已管理级【L2】	需求管理、项目计划、配置管理、项目监督	项目级可重复

	与控制、供应商合同管理、度量和分析、过程和产品质量保证	
混乱级（初始级） 【L1】		

39、嵌入式微处理器体系结构

体系结构分类	定义	特点	典型应用
冯·诺依曼结构	冯·诺依曼结构也称普林斯顿结构，是一种将程序指令存储器和数据存储器合并在一起的存储器结构。	指令与数据存储器合并在一起。 指令与数据都通过相同的数据总线传输。	一般用于 PC 处理器，如 I3、I5、I7 处理器。 注：常规计算机属于冯·诺依曼结构
哈佛结构	哈佛结构是一种并行体系结构，它的主要特点是将程序和数据存储在不同的存储空间中，即程序存储器和数据存储器是两个独立的存储器，每个存储器独立编址、独立访问。	指令与数据分开存储，可以并行读取，有较高的数据吞吐率。 有4条总线：指令和数据的数据总线与地址总线。	一般用于嵌入式系统处理器。 注：DSP 属于哈佛结构

40、软件架构风格

架构风格定义了用于描述系统的术语表和一组指导构建系统的规则

五大架构风格	子风格
数据流风格 【Data Flow】	批处理【Batch Sequential】、管道-过滤器【Pipes and Filters】
调用/返回风格 【Call/Return】	主程序/子程序【Main Program and Subroutine】 面向对象【Object-oriented】、分层架构【Layered System】
独立构件风格 【Independent Components】	进程通信【Communicating Processes】、 事件驱动系统（隐式调用）【Event system】
虚拟机风格 【Virtual Machine】	解释器【interpreter】、规则系统【Rule-based System】

以数据为中心 【Data-centered】	数据库系统【Database System】、黑板系统【Blackboard System】、超文本系统【Hypertext System】
---------------------------	--

41、云原生架构风格

(1) 云计算基本概念：

云计算是集合了大量计算设备和资源，对用户屏蔽底层差异的分布式处理架构，其用户与提供实际服务的计算资源是相分离的。

云计算优点：超大规模、虚拟化、高可靠性、高可伸缩性、按需服务、成本低【前期投入低、综合使用成本也低】。

(2) 分类

按服务类型分类：

SaaS【软件即服务】	基于多租户技术实现，直接提供应用程序
PaaS【平台即服务】	虚拟中间件服务器、运行环境和操作系统
IaaS【基础设施即服务】	包括服务器、存储和网络等服务

按部署方式分类：

公有云：面向互联网用户需求，通过开放网络提供云计算服务

私有云：面向企业内部提供云计算服务

混合云：兼顾以上两种情况的云计算服务

42、微服务与 SOA 对比

微服务	SOA
能拆分的就拆分	是整体的，服务能放一起的都放一起
纵向业务划分	是水平分多层
由单一组织负责	按层级划分不同部门的组织负责
细粒度	粗粒度
两句话可以解释明白	几百字只相当于 SOA 的目录
独立的子公司	类似大公司里面划分了一些业务单元（BU）
组件小	存在较复杂的组件
业务逻辑存在于每一个服务中	业务逻辑横跨多个业务领域

使用轻量级的通信方式，如 HTTP	企业服务总线（ESB）充当了服务之间通信的角色
-------------------	-------------------------

43、微服务

（1）什么是微服务

微服务顾名思义，就是很小的服务，所以它属于面向服务架构的一种。

（2）微服务的优势

优点	解读
【复杂应用解耦】	小服务（且专注于做一件事情） 化整为零，易于小团队开发
【独立】	独立开发 独立测试及独立部署（简单部署） 独立运行（每个服务运行在其独立进程中）
【技术选型灵活】	支持异构（如：每个服务使用不同数据库）
【容错】	故障被隔离在单个服务中，通过重试、平稳退化等机制实现应用层容错
【松耦合，易扩展】	可根据需求独立扩展

（3）微服务面临的挑战

分布式环境下的数据一致性【更复杂】

测试的复杂性【服务间依赖测试】

运维的复杂性

44、数据中台必备的 4 个核心能力

（1）数据汇聚整合能力

（2）数据提纯加工能力

（3）数据服务可视化

（4）价值变现方面

45、MVC

Model（模型）是应用程序中用于处理应用程序数据逻辑的部分。通常模型对象负责在数据库中存取数据。

View（视图）是应用程序中处理数据显示的部分。通常视图是依据模型数据创建的。

Controller（控制器）是应用程序中处理用户交互的部分。通常控制器负责从视图读取数据，

控制用户输入，并向模型发送数据。

J2EE 体系结构中：

视图 (View)：JSP

控制 (Controller)：Servlet

模型 (Model)：Entity Bean、Session Bean

46、数据库故障与恢复

故障关系	故障原因	解决方法
事务本身的可预期故障	本身逻辑	在程序中预先设置 Rollback 语句
事务本身的不可预期故障	算术溢出、违反存储保护	由 DBMS 的恢复子系统通过日志,撤消事务对数据库的修改,回退到事务初始状态
系统故障	系统停止运转	通常使用检查点法
介质故障	外存被破坏	一般使用日志重做业务

47、虚拟机技术与容器技术的对比

对比项	虚拟机技术	容器技术
镜像大小	包含 GuestOS, G 量级以上	仅包含运行的 Bin/Lib, M 量级
资源要求	CPU 与内存按核、按 G 分配	CPU 与内存按单核、低于 G 量级分配
启动时间	分钟级	毫秒级
可持续性	跨物理机迁移	跨操作系统平台迁移
弹性伸缩	VM 伸缩, CPU/内存手动伸缩	实例自动伸缩、CPU 内存自动在线伸缩
隔离策略	操作系统、系统级别	Cgroups, 进程级别

48、边云协同的分类

【资源协同】边缘节点有基础设施资源的调度管理能力，可与云端协同。

【数据协同】边缘节点采集数据并初步分析，再发给云端做进一步处理。

【智能协同】分布式智能，云端做集中式模型训练，再将模型下发到边缘节点。

【应用管理协同】边缘节点提供应用部署与运行环境，云端主要提供应用开发、测试环境。

【业务管理协同】边缘节点提供模块化、微服务化的应用/数字孪生/网络等应用实例；云端主要提供按照客户需求实现应用/数字孪生/网络等的业务编排能力。

【服务协同】边缘节点按照云端策略实现部分 ECSaaS 服务，通过 ECSaaS 与云端 SaaS 的协同实现面向客户的按需 SaaS 服务；云端主要提供 SaaS 服务在云端和边缘节点的服务

分布策略，以及云端承担的 SaaS 服务能力。

49、负载均衡

负载均衡技术：

(1) 应用层负载均衡：http 重定向、反向代理服务器；(2) 传输层负载均衡：DNS 域名解析负载均衡、基于 NAT 的负载均衡；(3) 硬件负载均衡：F5；(4) 软件负载均衡：LVS、Nginx、HAproxy。

负载均衡算法：

静态算法（不考虑动态负载）：

轮转算法：轮流将服务请求（任务）调度给不同的节点（即：服务器）。

加权轮转算法：考虑不同节点处理能力的差异。

源地址哈希散列算法：根据请求的源 IP 地址，作为散列键从静态分配的散列表找出对应的节点。

50、软件架构复用

【软件复用】是一种系统化的软件开发过程，通过识别、分析、分类、获取和修改软件实体，以便在不同软件开发过程中重复使用它们。

软件开发过程中重复使用相同或相似【软件元素】的过程。

软件架构	机会复用	开发过程中，只要发现有可复用资产，就对其进行复用
复用类型	系统复用	开发之前，要进行规划，以决定哪些需要复用

软件架构复用的基本过程：复用的基本过程主要包括 3 个阶段：首先构造/获取可复用的软件资产，其次管理这些资产，最后针对特定的需求，从这些资产中选择可复用的部分，以开发满足需求的应用系统。

可复用资产范围：需求、架构设计、元素、建模与分析、测试、项目规划、过程、方法和工具、人员、样本系统、缺陷消除。

51、Redis 与 Memcache 能力比较

工作	MemCache	Redis
数据类型	简单 key/value 结构	丰富的数据结构
持久性	不支持	支持
分布式存储	客户端哈希分片/一致性哈希	多种方式，主从、Sentinel、Cluster 等

多线程支持	支持	支持（Redis5.0 及以前版本不支持）
内存管理	私有内存池/内存池	无
事务支持	不支持	有限支持
数据容灾	不支持，不能做数据恢复	支持，可以在灾难发生时，恢复数据

52、Redis 集群切片的常见方式

集群切片方式	核心特点
客户端分片	在客户端通过 key 的 hash 值对应到不同的服务器。
中间件实现分片	在应用软件和 Redis 中间，例如：Twemproxy、Codis 等，由中间件实现服务到后台 Redis 节点的路由分派。
客户端服务端协作分片	Redis Cluster 模式，客户端可采用一致性哈希，服务端提供节点的重定向到 slot 上。不同的 slot 对应到不同服务器。

53、Redis 分布式存储方案

分布式存储方案	核心特点
主从（Master/Slave）模式	一主多从，故障时手动切换。
哨兵（Sentinel）模式	有哨兵的一主多从，主节点故障自动选择新的主节点。
集群（Cluster）模式	分节点对等集群，分 slots，不同 slots 的信息存储到不同节点。

54、Redis 数据分片方案

分片方案	分片方式	说明
范围分片	按数据范围值来做分片	例：按用户编号分片，0-999999 映射到实例 A；1000000-1999999 映射到实例 B。
哈希分片	通过对 key 进行 hash 运算分片	可以把数据分配到不同实例，这类似于取余操作，余数相同的，放在一个实例上。

一致性哈希分片	哈希分片的改进	可以有效解决重新分配节点带来的无法命中问题。
---------	---------	------------------------

55、JWT

JWT (JSON Web Token) 是一种用于身份验证和授权的开放标准。

它是一种轻量级的、基于 JSON 的令牌，可以在客户端和服务端之间传递信息。

JWT 由三部分组成：

【头部】包含令牌类型和所使用的算法

【载荷】包含用户信息和其他元数据

【签名】用于验证令牌的完整性和真实性

JWT 的应用场景：【信息交换】【授权】

不需要服务器端存储状态，安全地传递【非敏感信息】

56、REST 概念

REST (Representational State Transfer, 表述性状态转移) 是一种通常使用 HTTP 和 XML 进行基于 Web 通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

REST 的五个原则：网络上的所有事物都被抽象为资源；每个资源对应一个唯一的资源标识；通过通用的连接件接口对资源进行操作；对资源的各种操作不会改变资源标识；所有的操作都是无状态的。

57、响应式 Web 设计

响应式 WEB 设计是一种网络页面设计布局，其理念是：集中创建页面的图片排版大小，可以智能地根据用户行为以及使用的设备环境进行相对应的布局。方法：采用流式布局和弹性化设计、响应式图片。

58、ADL

ADL 是这样一种形式化语言，它在底层语义模型的支持下，为软件系统的概念体系结构建模提供了具体语法和概念框架。

C2SADL 【基于组件和消息的软件架构描述语言】

Wright 【分布、并发类型的架构描述语言】

ACME 【架构互换语言】

UniCon 【基于组件和连接的架构描述语言】

Rapide【基于事件的架构描述语言】

其他【Darwin、MetaH、Aesop、Weaves、SADL、xADL】(2) ADL 的三个基本元素

ADL 的三个基本元素：

构件：计算或数据存储单元；

连接件：用于构件之间交互建模的体系结构构造块及其支配这些交互的规则；

架构配置：描述体系结构的构件与连接件的连接图。

59、反规范化的技术手段以及优缺点

技术手段	说明
增加派生性冗余列	已有单价和数量列，增加“总价”列
增加冗余列	已有学号列，增加“姓名”列
重新组表	把拆分的表重新组表
分割表	把用户表做水平分割，长沙的用户存在长沙，上海的用户存在上海

反规范化的优点：连接操作少，检索快、统计快；需要查的表减少，检索容易。

反规范化的缺点	解决方案
数据冗余，需要更大存储空间	无解
插入、更新、删除操作开销更大	无解
数据不一致 可能产生添加、修改、删除异常	1、触发器数据同步 2、应用程序数据同步 3、批处理
更新和插入代码更难写	无解

60、视图的优点

- (1) 视图能简化用户的操作
- (2) 视图机制可以使用户以不同的方式查询同一数据
- (3) 视图对数据库重构提供了一定程度的逻辑独立性
- (4) 视图可以对机密的数据提供安全保护

其中物化视图：将视图的内容物理存储起来，其数据随原始表变化，同步更新。

61、分表和分区

	分区	分表
共性	1、都针对数据表 2、都使用了分布式存储 3、都提升了查询效率 4、都降低了数据库的频繁 I/O 压力值	
差异	逻辑上还是一张表	逻辑上已是多张表。

62、关系型数据库和 NoSQL 对比

对比维度	关系数据库	NoSQL
应用领域	面向通用领域	特定应用领域
数据容量	有限数据	海量数据
数据类型	结构化数据【二维表】	非结构化数据
并发支持	支持并发、但性能低	高并发
事务支持	高事务性	弱事务性
扩展方式	向上扩展	向外扩展

63、信息安全的 5 个基本要素

名称	说明
机密性	机密性是指网络信息不泄漏给非授权的用户、实体或程序，能够防止非授权者获取信息。
完整性	完整性是指网络信息或系统未经授权不能进行更改的特性。
可用性	可用性是指合法许可的用户能够及时获取网络信息或服务的特性。
可控性	可控性是指可以控制授权范围内的信息流向及行为方式。
可审查性	可审查性是指对出现的信息安全问题提供调查的依据和手段。

64、国产密码算法

算法名称	算法特性描述	备注
SM1	对称加密，分组长度和密钥长度都为 128 比特	广泛应用于电子政务、电子商务及国民经济的各个应用领域

SM2	非对称加密，用于公钥加密算法、密钥交换协议、数字签名算法	国家标准推荐使用素数域 256 位椭圆曲线
SM3	杂凑算法，杂凑值长度为 256 比特	适用于商用密码应用中的数字签名和验证
SM4	对称加密，分组长度和密钥长度都为 128 比特	适用于无线局域网产品
SM9	标识密码算法	不需要申请数字证书，适用于互联网应用的各种新兴应用的安全保障

65、BLP 模型

Bell-LaPadula 模型是符合军事安全策略的计算机安全模型，简称 BLP 模型。BLP 模型的安全规则如下：

简单安全规则：安全级别低的主体不能读安全级别高的客体；

星属性安全规则：安全级别高的主体不能往低级别的客体写；

强星属性安全规则：不允许对另一级别进行读写；

自主安全规则：使用访问控制矩阵来定义说明自由存取控制。

66、Biba 模型

BiBa 模型主要用于防止非授权修改系统信息，以保护系统的信息完整性。该模型同 BLP 模型类似，采用主体、客体、完整性级别描述安全策略要求。BiBa 模型能够防止数据从低完整性级别流向高完整性级别，其安全规则如下：

星完整性规则：表示完整性级别低的主体不能对完整性级别高的客体写数据；

简单完整性规则：表示完整性级别高的主体不能从完整性级别低的客体读取数据；

调用属性规则：表示一个完整性级别低的主体不能从完整性级别高的客体调用程序或服务。

67、Chinese Wall 模型

Chinese Wall 模型的安全策略的基础是客户访问的信息不会与当前他们可支配的信息产生冲突。其访问客体控制的安全规则如下：

与主体曾经访问过的信息属于同一公司数据集合的信息，即墙内信息可以访问；

属于一个完全不同的利益冲突组的可以访问；

主体能够对一个客体进行写的前提是主体未对任何属于其他公司数据集进行过访问。

定理 1：一个主体一旦访问过一个客体，则该主体只能访问位于同一公司数据集的客体或在

不同利益组的客体。

定理 2：在一个利益冲突组中，一个主体最多只能访问一个公司数据集。

68、WPDRRC 模型

WPDRRC 模型：WPDRRC 信息安全模型是我国“八六三”信息安全专家组提出的适合中国国情的信息系统安全保障体系建设模型。

WPDRRC 模型包括 6 个环节和 3 大要素。

6 个环节包括：预警、保护、检测、响应、恢复和反击。模型蕴涵的网络安全能力主要是预警能力、保护能力、检测能力、响应能力、恢复能力和反击能力。

3 大要素包括人员、策略和技术。

69、被动攻击

被动攻击：收集信息为主，破坏保密性。

攻击类型	攻击名称	描述
被动攻击	窃听（网络监听）	用各种可能的合法或非法的手段窃取系统中的信息资源和敏感信息。
	业务流分析	通过对系统进行长期监听，利用统计分析方法对诸如通信频度、通信的信息流向、通信总量的变化等参数进行研究，从而发现有价值的信息和规律。
	非法登录	有些资料将这种方式归为被动攻击方式。

70、主动攻击

主动攻击的类别主要有：中断（破坏可用性），篡改（破坏完整性），伪造（破坏真实性）

攻击类型	攻击名称	描述
主动攻击	假冒身份	通过欺骗通信系统（或用户）达到非法用户冒充成为合法用户，或者特权小的用户冒充成为特权大的用户的目的。黑客大多是采用假冒进行攻击。
	抵赖	这是一种来自用户的攻击，比如：否认自己曾经发布过的某条消息、伪造一份对方来信等。
	旁路控制 【旁路攻	密码学中是指绕过对加密算法的繁琐分析，利用密码算法的硬件实现的运算中泄漏的信息。如执行时间、功耗、电磁辐射等，结

击】	合统计理论快速的破解密码系统。
重放攻击	所截获的某次合法的通信数据拷贝，出于非法的目的而被重新发送。 加时间戳能识别并应对重放攻击。
拒绝服务 (DOS)	对信息或其它资源的合法访问被无条件地阻止。
XSS 跨站脚本攻击	通过利用网页【开发时留下的漏洞】，通过巧妙的方法注入恶意指令代码到网页
CSRF 跨站请求伪造攻击	攻击者通过一些技术手段欺骗用户的浏览器与访问一个自己曾经认证过的网站并执行一些操作（如转账或购买商品等）。
缓冲区溢出攻击	利用【缓冲区溢出漏洞】所进行的攻击。在各种操作系统、应用软件中广泛存在。

71、访问控制类型

控制类型	说明
基于对象的访问控制	OBAC 访问控制系统是从信息系统的差异变化和用户需求出发，有效地解决了信息数据量大、数据种类繁多、数据更新变化频繁的大型管理信息系统的的核心所在。
基于角色的访问控制	RBAC 就是指根据完成某些职任务所需要的访问权限来进行授权和管理。 RBAC 由用户(U)、角色(R)、会话(S)和权限(P)四个基本要素组成。
基于任务的访问控制	TBAC 是从应用和企业层角度来解决安全问题，以面向任务的观点，从任务（活动）的角度来建立安全模型和实现安全机制，在任务处理的过程中提供动态实时的安全管理。 TBAC 模型由工作流、授权结构体、受托人集和许可集四部分组成。 其一般用五元组（S，O，P，L，AS）来表示，其中 S 表示主体，O 表示客体，P 表示许可，L 表示生命期，AS 表示授权步。
基于属性的访问控制	ABAC 是根据主体的属性、客体的属性、环境的条件以及访问策略对主体的请求操作进行授权许可或拒绝。

72、Redis 持久化

RDB：传统数据库中快照的思想。指定时间间隔将数据进行快照存储。

AOF：传统数据库中日志的思想，把每条改变数据集的命令追加到 AOF 文件末尾，这样出了问题了，可以重新执行 AOF 文件中的命令来重建数据集。

对比维度	RDB 持久化	AOF 持久化
备份量	重量级的全量备份，保存整个数据库	轻量级增量备份，一次只保存一个修改命令
保存间隔时间	保存间隔时间长	保存间隔时间短，默认 1 秒
还原速度	数据还原速度快	数据还原速度慢
阻塞情况	save 会阻塞，但 bgsave 或者自动不会阻塞	无论是平时还是 AOF 重写，都不会阻塞
数据体积	同等数据体积：小	同等数据体积：大
安全性	数据安全性：低，容易丢数据	数据安全性：高，根据策略决定

73、安全保护等级

	公民、法人和其它组织权益	社会秩序和公共利益	国家安全
用户自主保护级	损害		
系统审计保护级	严重损害	损害	
安全标记保护级		严重损害	损害
结构化保护级			严重损害
访问验证保护级			特别严重损害

74、恢复块方法与 N 版本程序设计对比

	恢复块方法	N 版本程序设计
硬件运行环境	单机	多机
错误检测方法	验证测试程序	表决
恢复策略	后向恢复	前向恢复
实时性	差	好

75、Session 有状态和无状态问题

无状态服务（stateless service）对单次请求的处理，不依赖其他请求，也就是说，处理一次请求所需的全部信息，要么都包含在这个请求里，要么可以从外部获取到（比如说数据库），服务器本身不存储任何信息。

有状态服务（stateful service）则相反，它会在自身保存一些数据，先后的请求是有关联的。

76、性能评估

方法	描述	特点
时钟频率法	以时钟频率高低衡量速度	仅考虑 CPU
指令执行速度法	表示机器运算速度的单位是 MIPS	仅考虑 CPU
等效指令速度法 (吉普森混合法)	通过各类指令在程序中所占的比例 (Wi) 进行计算得到的。	仅考虑 CPU, 综合考虑指令比例不同的问题。
数据处理速率法 (PDR)	PDR 值的方法来衡量机器性能, PDR 值越大, 机器性能越好。	$PDR = L/R$ 仅考虑 CPU+存储
综合理论性能法 (CTP)	CTP 用 MTOPS 表示。CTP 的估算方法是, 首先算出处理部件每个计算单元的有效计算率, 再按不同字长加以调整, 得出该计算单元的理论性能, 所有组成该处理部件的计算单元的理论性能之和即为 CTP。	仅考虑 CPU+存储
基准程序法	把应用程序中用得最多、最频繁的那部分核心程序作为评估计算机系统性能的标准程序, 称为基准测试程序 (benchmark)。	综合考虑多部分, 基准程序法是目前一致承认的测试系统性能的较好方法。

77、性能指标

分类		指标
硬件	计算机	时钟频率 (主频); 运算速度与精度; 内存的存储容量; 存储器的存取周期; 数据处理速率 PDR; 吞吐率; 各种响应时间; 各种利用率; RASIS 特性; 平均故障响应时间; 兼容性; 可扩充性; 性能价格比。
	路由	设备吞吐量、端口吞吐量、全双工线速转发能力、丢包率、时延、时延抖动、

	器	VPN 支持能力、端口硬件队列数、基于 Web 的管理、网管类型等。
	交换机	交换机类型、配置、支持的网络类型、最大 ATM 端口数、支持协议和标准等。
	网络	设备级性能指标；网络级性能指标；应用级性能指标；用户级性能指标；吞吐量。
软件	操作系统	系统的可靠性、系统的吞吐率（量）、系统响应时间、系统资源利用率、可移植性。
	数据库管理系统	衡量数据库管理系统的主要性能指标包括数据库本身和管理系统两部分，有：数据库的大小、数据库中表的数量、单个表的大小、表中允许的记录（行）数量、单个记录（行）的大小、表上所允许的索引数量、数据库所允许的索引数量、最大并发事务处理能力、负载均衡能力、最大连接数等。
	WEB 服务器	最大并发连接数、响应延迟、吞吐量。

78、嵌入式数据库分类

按照数据库存储位置的不同而进行分类是目前广泛采用的分类方法，它可以划分为三类：

基于内存方式（Main Memory Database，MMDB）

基于文件方式（File Database，FDB）

基于网络方式（Netware Database，NDB）

	定义	特点
基于内存的数据库 MMDB	基于内存的数据库系统是实时系统和数据库系统的有机结合。	内存数据库是支持实时事务的最佳技术，其本质特征是以其“主拷贝”或“工作版本”常驻内存，即活动事务只与实时内存数据库的内存拷贝打交道。
基于文件的数据库 FDB	基于文件的数据库系统就是以文件方式存储数据库数据，即数据按照一定格式储存在磁盘中。使用时由应用程序通过相应的驱动程序甚至直接对数据文件进行读写。	这种数据库的访问方式是被动式的，只要了解其文件格式，任何程序都可以直接读取，因此它的安全性很低。虽然文件数据库存在诸多弊端，但是，可以满足嵌入式系统在空间、时间等方面的特

		殊要求。
基于网络的数据库 NDB	基于网络的数据库系统是基于手机 4G/5G 的移动通信基础之上的数据库系统，在逻辑上可以把嵌入式设备看作远程服务器的一个客户端。	嵌入式网络数据库系统的特点是： 无需解析 SQL 语句；支持更多的 SQL 操作；客户端小、无需支持可剪裁性；有利于代码重用。

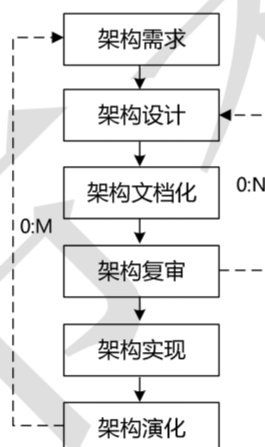
79、基于架构的软件设计（ABSD）

（ABSD 方法是架构驱动，即强调由业务【商业】、质量和功能需求的组合驱动架构设计。ABSD 方法有三个基础。第一个基础是功能的分解。在功能分解中，ABSD 方法使用已有的基于模块的内聚和耦合技术；第二个基础是通过选择架构风格来实现质量和业务需求；第三个基础是软件模板的使用。软件模板利用了一些软件系统的结构。

视角与视图：从不同的视角来检查，所以会有不同的视图。

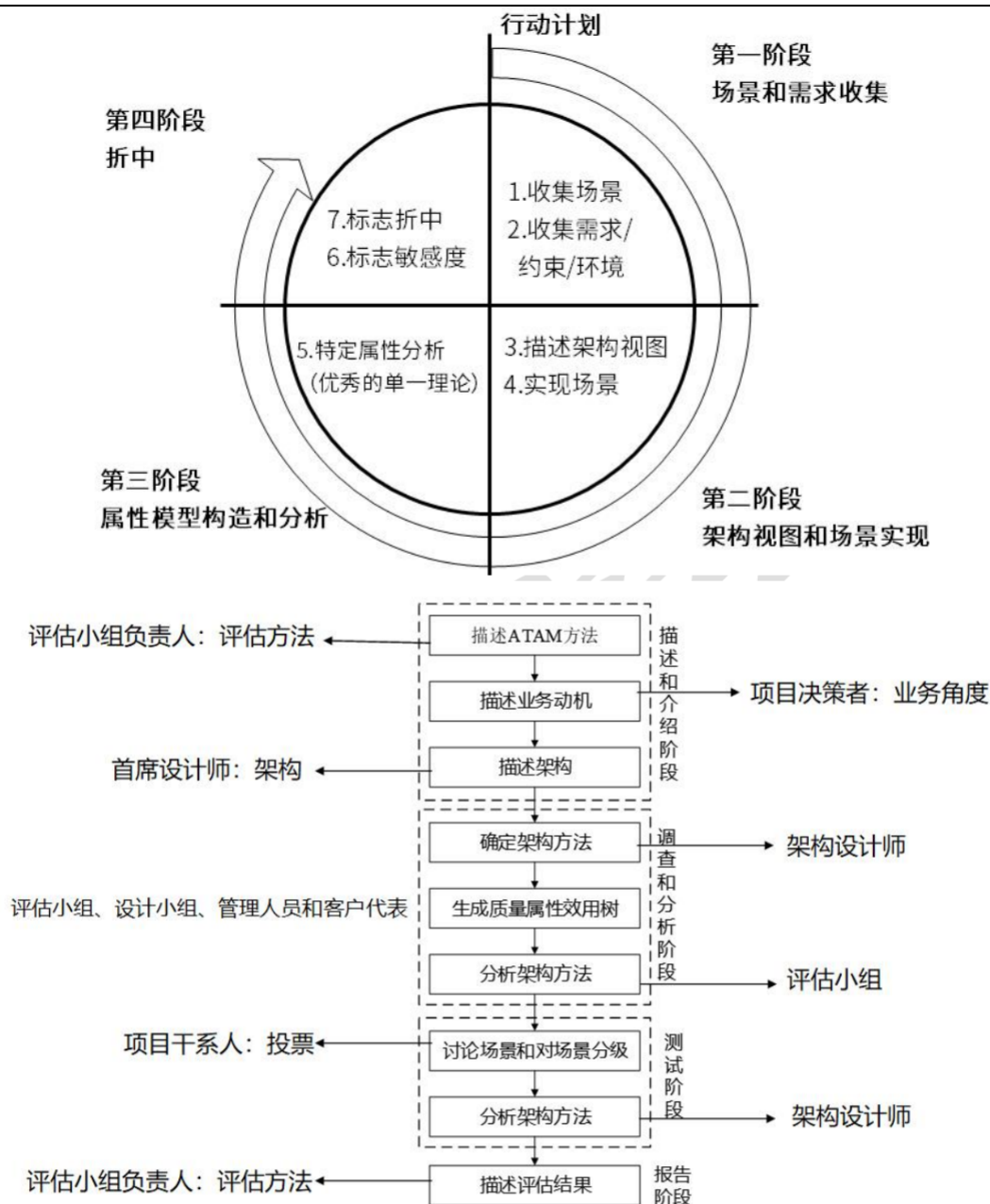
用例用来捕获功能需求、特定场景【刺激、环境、响应】用来捕获质量需求。

开发过程：



80、架构权衡分析法（ATAM）

（在 SAAM 的基础上发展起来的，主要针对性能、实用性、安全性和可修改性，在系统开发之前，对这些质量属性进行评价和折中。）



81、风险点与非风险点、敏感点与权衡点

风险点：系统架构风险是指架构设计中潜在的、存在问题的架构决策所带来的隐患。

非风险点：是指不会带来隐患，一般以“XXX 要求是可以实现【或接受】的”方式表达。

敏感点：敏感点是一个或多个构件（和/或构件之间的关系）的特性，它能影响系统的某个质量属性。

权衡点：影响多个质量属性的特性，是多个质量属性的敏感点。

82、场景的描述

场景是从风险承担者的角度与系统交互的简短描述。场景可从六个方面进行描述：刺激源、刺激、制品、环境、响应、响应度量。

刺激源 (Source)：这是某个生成该刺激的实体（人、计算机系统或者任何其他刺激器）。

刺激 (Stimulus)：该刺激是当刺激到达系统时需要考虑的条件。

环境 (Environment)：该刺激在某些条件内发生。当激励发生时，系统可能处于过载、运行或者其他情况。

制品 (Artifact)：某个制品被激励。这可能是整个系统（或系统的一部分）。

响应 (Response)：该响应是在激励到达后所采取的行动。

响应度量 (Measurement)：当响应发生时，应当能够以某种方式对其进行度量，以对需求进行测试。

83、开发期质量属性和运行期质量属性：

开发期质量属性主要指在软件开发阶段所关注的质量属性，主要包含 6 个方面：

易理解性：指设计被开发人员理解的难易程度。

可扩展性：软件因适应新需求或需求变化而增加新功能的能力，也称为灵活性。

可重用性：指重用软件系统或某一部分的难易程度。

可测试性：对软件测试以证明其满足需求规范的难易程度。

可维护性：当需要修改缺陷、增加功能、提高质量属性时，识别修改点并实施修改的难易程度。

可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度。

运行期质量属性主要指在软件运行阶段所关注的质量属性，主要包含 7 个方面：

性能：性能是指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。

安全性：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。

可伸缩性：指当用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。

互操作性：指本软件系统与其他系统交换数据和相互调用服务的难易程度。

可靠性：软件系统在一定的时间内持续无故障运行的能力。

可用性：指系统在一定时间内正常工作的时间所占的比例。可用性会受到系统错误，恶意攻击，高负载等问题的影响。

鲁棒性：是指软件系统在非正常情况(如用户进行了非法操作、相关的软硬件系统发生了故障等)下仍能够正常运行的能力，也称健壮性或容错性。

84、常考质量属性以及代表参数和设计策略

(1) 性能

性能 (performance) 是指系统的响应能力，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的个数。例如：a.同时支持 1000 并发；b.响应时间小于 1s；c.显示分辨率达到 4K。

代表参数：响应时间、吞吐量 设计策略：优先级队列、资源调度

(2) 可用性

可用性 (availability) 是系统能够正常运行的时间比例。经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。例如：a.主服务器故障，1 分钟内切换至备用服务器；b.系统故障，1 小时内修复；c.系统支持 7×24 小时工作。

代表参数：故障间隔时间 设计策略：冗余、心跳

(3) 安全性

安全性 (security) 是指系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力。安全性又可划分为机密性【信息不泄漏给未授权的用户】、完整性【防止信息被篡改】、不可否认性【不可抵赖】及可控性【对信息的传播及内容具有控制的能力】等特性。例如：a.可抵御 SQL 注入攻击；b.对计算机的操作都有完整记录；c.用户信息数据库授权必须保证 99.9%可用。

设计策略：追踪审计

(4) 可修改性

可修改性 (modifiability) 是指能够快速地对较高的性能价格比系统进行变更的能力。通常以某些具体的变更为基础，通过考察这些变更的代价衡量可修改性。(可扩展性与之相近) 例如：a.更改系统报表模块，必须在 2 人周内完成；b.对 Web 界面风格进行修改，修改必须在 4 人月内完成。

主要策略：信息隐藏 (二义性：良好的封装能够做到信息隐藏，一般归于可修改性策略；信息隐藏也能够体现在安全性当中)

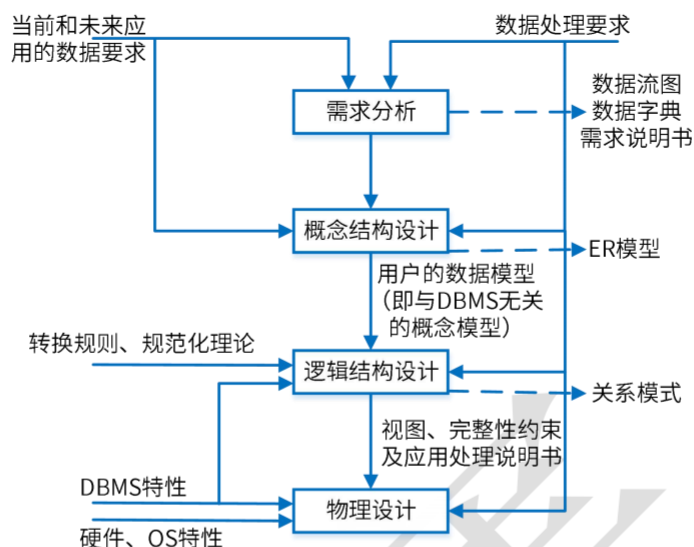
(5) 易用性

易用性关注的是对用户来说完成某个期望任务的容易程度和系统所提供的用户支持的种类。例如：a.界面友好；b.新用户学习使用系统时间不超过 2 小时。

(6) 可测试性

软件可测试性是指通过测试揭示软件缺陷的容易程度。例如：a.提供远程调试接口，支持远程调试。

85、数据库设计过程



86、范式

第一范式（1NF）：在关系模式 R 中，当且仅当所有域只包含原子值，即每个属性都是不可再分的数据项，则称关系模式 R 是第一范式。

第二范式（2NF）--消除非主属性对码的部分函数依赖：当且仅当关系模式 R 是第一范式（1NF），且每一个非主属性完全依赖候选键（没有不完全依赖）时，则称关系模式 R 是第二范式。

第三范式（3NF）--消除非主属性对码的传递函数依赖：当且仅当关系模式 R 是第二范式（2NF），且 R 中没有非主属性传递依赖于候选键时，则称关系模式 R 是第三范式。

BC 范式（BCNF）：设 R 是一个关系模式，F 是它的依赖集，R 属于 BCNF 当且仅当其 F 中每个依赖的决定因素必定包含 R 的某个候选码。

（可以理解为在 3NF 基础上，消除主属性之间的传递函数依赖和部分函数依赖）

第四范式（4NF）是限制关系模式的属性间不允许有非平凡且非函数依赖的多值依赖。

87、NoSQL 分类

分类	典型应用场景	数据模型	优点	缺点	举例
键值 (key-value)	内容缓存, 主要用于处理大量数据的高访问负载, 也用于一些日志系统等	Key 指向 Value 的键值对, 通常用 hash table 来实	查找速度快	数据无结构化, 通常只被当作字符串或者二进	Redis, Tokyo Cabinet/ Tyrant, Voldemort, Oracle BDB

	等。	现		制数据	
列存储数据库	分布式的文件系统	以列簇式存储，将同一列数据存在一起	查找速度快，可扩展性强，更容易进行分布式扩展	功能相对局限	HBase, Cassandra, Riak
文档型数据库	Web 应用（与 Key-Value 类似，Value 是结构化的，不同的是数据库能够了解 Value 的内容）	Key-Value 对应的键值对，Value 为结构化数据	数据结构要求不严格，表结构可变，不需要像关系型数据库一样需要预先定义表结构	查询性能不高，而且缺乏统一的查询语法。	CouchDB, MongoDb
图形数据库（Graph）	社交网络，推荐系统等。专注于构建关系图谱	图结构	利用图结构相关算法。比如最短路径寻址，N 度关系查找等	很多时候需要对整个图做计算才能得出需要的信息，而且这种结构不太好做分布式的集群方案。	Neo4J, InfoGrid, Infinite Graph

88、封锁协议

一级封锁协议。事务 T 在修改数据 R 之前必须先对其加 X 锁，直到事务结束才释放。可防止丢失修改。

二级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁，读完后即可释放 S 锁。可防止丢失修改，还可防止读“脏”数据。

三级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁，直到事务结束

才释放。可防止丢失修改、防止读“脏”数据与防止数据重复读。

两段锁协议。可串行化的。可能发生死锁。

89、备份

(1) 冷备份也称为静态备份，是将数据库正常关闭，在停止状态下，将数据库的文件全部备份（复制）下来。

(2) 热备份也称为动态备份，是利用备份软件，在数据库正常运行的状态下，将数据库中的数据文件备份出来。

(3) 完全备份：备份所有数据。

(4) 增量备份：仅备份上一次完全备份之后变化的数据。

(5) 增量备份：备份上一次备份之后变化的数据。

(6) 日志文件：事务日志是针对数据库改变所做的记录，它可以记录针对数据库的任何操作，并将记录结果保存在独立的文件中。

90、TCP 与 UDP

	TCP	UDP
共同点	基于 IP 协议的传输层协议，可以端口寻址	
不同点	面向连接（连接管理）、三次握手、流量控制、差错校验和重传、IP 数据报按序接收不丢失不重复、可靠性强、牺牲通信量、效率低	不可靠、无连接、错误检测功能弱，无拥塞控制、无流量控制，有助于提高传输的高速率性。 不对无序 IP 数据报重新排序、不负责重传、不消除重复 IP 数据报、不对已收到的数据报进行确认、不负责建立或终止连接，这些由 UDP 进行通信的应用程序进行处理。
相关协议	HTTP、FTP、Telnet、POP3、SMTP	DNS、DHCP、TFTP、SNMP

91、TCP/IP 协议簇

SMTP：25 端口，邮件发送

FTP：20 数据端口/21 控制端口，文件传输协议

HTTP：80 端口，超文本传输协议，网页传输

DHCP：67 端口，IP 地址自动分配

SNMP：161 端口，简单网络管理协议

DNS：53 端口，域名解析协议，记录域名与 IP 的映射关系

TCP：可靠的传输层协议

UDP：不可靠的传输层协议

ICMP：因特网控制协议，PING 命令来自该协议

IGMP：组播协议

ARP：地址解析协议，IP 地址转换为 MAC 地址

RARP：反向地址解析协议，MAC 地址转 IP 地址

92、网络分类

(1) 局域网 (LAN)

局域网是指在有限地理范围内将若干计算机通过传输介质互联成的计算机组（即通信网络），通过网络软件实现计算机之间的文件管理、应用软件共享、打印机共享、工作组内的日程安排、电子邮件和传真通信服务等功能。**局域网是封闭型的。**

(2) 城域网 (MAN)

(3) 广域网 (WAN)

(4) 因特网 (Internet)

其中局域网专用性非常强，具有比较稳定和规范的拓扑结构。常见的局域网拓扑结构有星状结构、树状结构、总线结构和环形结构。

93、5G 网络的主要特征

(1) 服务化架构：5G 核心网中引入了 SBA 服务化架构，实现网络功能的灵活定制和按需组合。

(2) 网络切片：通过网络切片技术在单个独立的物理网络上切分出多个逻辑网络，从而避免了为每一个服务建设一个专用的物理网络，极大地降低了建网和运维成本。

94、区块链技术

(1) 【区块链】≠ 比特币，比特币底层采用了区块链技术。比特币交易在我国定性为【非法运用】。

(2) 区块链的特点：

去中心化：由于使用分布式核算和存储，不存在中心化的硬件或管理机构，任意节点的权利和义务都是均等的，系统中的数据块由整个系统中具有维护功能的节点来共同维护。

开放性：系统是开放的，如：区块链上的【交易信息是公开的】，不过【账户身份信息是高度加密的】。

自治性：区块链采用基于协商一致的规范和协议（比如一套公开透明的算法）使得整个系统中的所有节点能够在去信任的环境自由安全的交换数据，使得对“人”的信任改成了对机器的信任，任何人为的干预不起作用。

安全性（信息不可篡改）：数据在多个节点存储了多份，篡改数据得改掉 51% 节点的数据，这太难。同时，还有其它安全机制，如：比特币的每笔交易，都由付款人用私钥签名，证明确实是他同意向某人付款，其它人无法伪造。

匿名性（去信任）：由于节点之间的交换遵循固定的算法，其数据交互是无需信任的（区块链中的程序规则会自行判断活动是否有效），因此交易对手无须通过公开身份的方式让对方自己产生信任，对信用的累积非常有帮助。

95、大数据技术

（1）大数据特点

4 个 V:

大规模 (Volume)

高速度 (Velocity)

多样化 (Variety)

价值密度低 (Value)

真实性 (Veracity) 【可选】

（2）大数据面临的问题

大数据面临着 5 个主要问题，分别是异构性 (Heterogeneity)、规模 (Scale)、时间性 (Timeliness)、复杂性 (Complexity) 和隐私性 (Privacy)。

（3）大数据分析步骤

大数据分析的分析步骤大致分为 5 个主要阶段：

数据获取/记录 【实时数据+离线数据】

信息抽取/清洗/注记

数据集成/聚集/表现

数据分析/建模

数据解释

96、Kappa 架构与 Lambda 的对比

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理，吞吐量大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱
使用场景	直接支持批处理，更适合对历史数据分析查询的场景，期望尽快得到分析结果，批处理可以更直接高效地满足这些需求。	不是 Lambda 的替代架构，而是简化，Kappa 放弃了对批处理的支持，更擅长业务本身为增量数据写入场景的分析需求
选择依据	根据两种架构对比分析，将业务需求、技术要求、系统复杂度、开发维护成本和历史数据处理能力作为选择考虑因素。 计算开销虽然存在一定差别，但是相差不是很大，所以不作为考虑因素。	

97、计算机访问磁盘存储的方式

（1）直连式存储（Direct Attached Storage,DAS）：计算机通过 I/O 端口直接访问存储设备的方式。

（2）网络连接的存储（Network Attached Storage,NAS）：计算机通过分布式文件系统访问存储设备的方式。

（3）存储区域网络（Storage Area Network,SAN）：计算机通过构建的独立存储网络访问存储设备的方式。

其中 SAN 和 NAS 都可以用于集中管理存储，并供多主机（服务器）共享存储。但是，NAS 通常是基于以太网，而 SAN 可使用以太网和光纤通道。此外，NAS 注重易用性、易管理性、可扩展性和更低的总拥有成本（TCO），而 SAN 则注重高性能和低延迟。

98、AI 芯片的关键特征

新型的计算范式：AI 计算既不脱离传统计算，也具有新的计算特质

训练和推断：AI 系统通常涉及训练和推断过程

大数据处理能力：满足高效能机器学习的数据处理要求

数据精度：降低精度的设计

可重构的能力：针对特定领域而不针对特定应用的设计，可以通过重新配置，适应新的 AI 算法、架构和任务

99、信息物理系统 CPS

CPS 的建设路径：CPS 体系设计->单元级 CPS 建设->系统级 CPS 建设->SoS 级 CPS 建设

级别	基本组成	说明	其他
单元级	单元级 CPS 是具备可感知、可计算、可交互、可延展、自决策功能的 CPS 最小单元 。	一个智能部件、一个工业机器人或一个智能机床都可能是一个 CPS 最小单元。	SoS 级 CPS 主要功能包括：数据存储、数据融合、分布式计算、大数据分析、数据服务，并在数据服务的基础上形成了资产性能管理和运营优化服务。 SoS 级 CPS 可以通过大数据平台，实现跨系统、跨平台的互联、互通和互操作，促成了多源异构数据的集成、交换和共享的闭环自动流动，在全局范围内实现信息全面感知、深度分析、科学决策和精准执行。
系统级	系统级 CPS 基于多个单元级 CPS 的状态感知、信息交互、实时分析，实现了局部制造资源的自组织、自配置、自决策、自优化。	多个最小单元(单元级)通过工业网络(如工业现场总线、工业以太网等),实现更大范围、更宽领域的自动流动，实现了多个单元级 CPS 的互联、互通和互操作，进一步提高制造资源优化配置的广度、深度和精度。	
SoS 级	多个系统级 CPS 的有机组合构成 SoS 级 CPS。	SoS 级 CPS 主要实现数据的汇聚，从而对内进行资产的优化和对外形成运营优化服务。	

100、数字孪生生态系统

行业应用层	包括智能制造、智慧城市在内的多方面应用
共性应用层	包括描述、诊断、预测、决策四个方面
模型构建与仿真分析层	包括数据建模、数据仿真和控制
数据互动层	包括数据采集、数据传输和数据处理等内容
基础支撑层	由具体的设备组成



免费订阅考试资讯



更多备考资料及学习福利
扫码添加希赛网课程顾问了解



免费在线刷题

制作于 24 年 7 月 适用于第 2 版教材